Answer-Q1:

```
/*
issues-date: 2026-09-26 22:30:00
commented by: Pham Ngoc Huy
data from public.dwd_order
- order_id: being duplicated from ORD008:
    # fix: 8,9,10
- user_id: user_id in dim_user sample only 1 -> 6 but data appear 6->8
    # fix: 1 -> 5

Note: all the [id key] -- is being wrong datatype at the beginning
- schema tells *_id: integer - but datasample: varchar
    # fix: remove all char "[A-z]"

database: PostgreSQL
*/

-- Question 1:
/*
Descrtions:
Write an SQL query:
    to classify buyers of each Level-1 category in the last 30 days into:
        - new-to-category
        - new-to-platform
    Also identify new-new users, meaning users whose first platform order and first category order happened

Display results:
    - category_name_level_1_id
    - buyer_type
    - users
*/
-- considered: if order happens, it will track -- no filtered in order 'cancel' or not.
with platform_firsts as (
-- objective: find very-first order time-stamp for each user accross the entire platform
select
user_id,
min(created_time) as first_platform_time
from public.dwd_order
where 1=1
group by
    user_id
),
category_firsts as (
-- objective: find very-first order time-stamp for each user within each specific category_name_level_1_id
select
user_id,
category_name_level_1_id,
min(created_time) as first_category_time
from dwd_order
where 1=1
group by
    user_id,
    category_name_level_1_id
),
classify_buyers as (
select
c.category_name_level_1_id,
c.user_id,
case
    -- happen in the same order --> new-new
    when c.first_category_time = p.first_platform_time then 'new-new'
    -- joined the platform in the last 30 days, but this category wasn't their first order
```

```
    when p.first_platform_time >= (select max(created_time) - interval '30 days' FROM public.dwd_order) THEN
-- Order-platform user buying in this category for the first time
else 'new-to-category'
end as buyer_type
from category_firsts c
join platform_firsts p
    on c.user_id = p.user_id
    -- filtered: last 30 days from 'buyers of each Level-1 category'
    and c.first_category_time >= (select max(created_time) - interval '30 days' from public.dwd_order)
)
select
category_name_level_1_id,
buyer_type,
count(distinct user_id) as users
from classify_buyers
group by
    category_name_level_1_id,
    buyer_type
order by
    category_name_level_1_id,
    buyer_type;
```

Answer-Q2:

```
/*
Question 2:
Write a SQL query to calculate 30-day conversion by user registration monthly cohort.
A user is considered converted if
    they place at least one completed order within 30 days after registration.

Display results:
    - reg_month
    - new_users
    - buyers_30d
    - conversion_30d
*/
with converted_users as (
select distinct
u.user_id
from public.dim_users u
join public.dwd_order o
    on u.user_id = o.user_id
        and upper(o.order_status) = 'COMPLETED'
        and o.created_time >= u.created_time
        and o.created_time <= u.created_time + interval '30 days'
)
select
to_char(u.created_time, 'YYYY-MM') as reg_month,
count(distinct u.user_id) as new_users,
count(distinct c.user_id) as buyers_30d,
round(
    count(distinct c.user_id) * 100.00 / nullif(count(distinct u.user_id), 0),
    2
) as conversion_30d
from public.dim_users u
left join converted_users c
    on u.user_id = c.user_id
group by
    to_char(u.created_time, 'YYYY-MM')
order by
    reg_month
;
```

Answer-Q3:

```
/*
Question 3:
Write a SQL query to identify users whose second purchase happened faster than 80% of other users.
Use only completed orders and calculate the time difference between the first and second purchase.

Display results:
- user_id
- first_order_time
- second_order_time
- minutes_between_orders
*/
with user_orders as (
-- Get: "Completed" Orders and the previous order-time
select
user_id,
lag(created_time) over (partition by user_id order by created_time) as first_order_time,
created_time as second_order_time,
row_number() over (partition by user_id order by created_time) as rn
from public.dwd_order
where 1=1
    and upper(order_status) = 'COMPLETED'
),
time_calculated as (
-- keep the second order only (rn=2) and calculat the minutes
select
user_id,
first_order_time,
second_order_time,
extract(epoch from (second_order_time - first_order_time)) / 60 as minutes_between_orders
from user_orders
where 1=1
    and rn = 2
),
percentile_ranked as (
select
user_id,
first_order_time,
second_order_time,
minutes_between_orders,
percent_rank() over (order by minutes_between_orders) as percent_rank
from time_calculated
)
-- Output: 20% users has the fastest
select
user_id,
first_order_time,
second_order_time,
round(minutes_between_orders,2) as minutes_between_orders
from percentile_ranked
where 1=1
    and percent_rank <= 0.20
order by
    minutes_between_orders asc;
```

Answer-Q4: